

CHECK POINT 01

Ferramentas de segurança no pipeline DevSecOps

njsscan | npm audit | Datree | StackHawk

GRUPO 3 • CLOUD SECURITY

Professor Fabio Pires

Leonardo Garroti • Aquiles Fonseca • Leandro de Souza

Equipe e objetivo

Integrantes

Leonardo Garroti

Aquiles Fonseca

Leandro de Souza

Objetivo do trabalho

Entender o que cada ferramenta verifica e mostrar como os alertas levam a correções no código e nas dependências.

Repositório público

[github.com/garrotii/
cp01-cloud-security-grupo-3](https://github.com/garrotii/cp01-cloud-security-grupo-3)

Agenda da apresentação

Tempo	Parte da apresentação
3 min	Fundamentos e pipeline
10 min	As quatro ferramentas do grupo
12 min	Laboratório guiado com SAST e SCA
3 min	Comparação e conclusão
2 min	Perguntas

Quatro frentes de segurança

Categoria	Ferramenta	O que verifica	Quando usar
SAST	njsscan	Código próprio	Desenvolvimento
SCA	npm audit	Dependências	Instalação e build
IaC	Datree	Arquivos Kubernetes	Antes do deploy
DAST	StackHawk	Aplicação ativa	Ambiente de teste

Cada ferramenta verifica uma parte diferente do projeto.

Testar cedo e conhecer os componentes

Shift-left

Trazer os testes de segurança para o começo do desenvolvimento.

- Corrigir antes de entregar
- Receber retorno no pipeline

SBOM

Lista dos componentes e das versões usados pela aplicação.

- Ajuda a localizar uma biblioteca
- Neste lab: formato CycloneDX

Testar cedo ajuda, mas os testes em execução continuam necessários.

Como organizamos o laboratório

Aplicação própria em Node.js

- Falhas escritas de propósito
- Versão insegura e versão corrigida
- Mesmo ambiente Docker
- Relatórios guardados no projeto

O que foi executado

- njsscan: SAST
- npm audit: SCA
- Datree e StackHawk: pesquisa
- Alvo local, sem testar sites de terceiros

A comparação usa resultados reais das duas versões do laboratório.

njsscan

Como analisa

- Procura padrões perigosos
- Usa libsast e Semgrep
- Não precisa iniciar a aplicação

Uso no laboratório

- Versão fixada: 1.0.0
- Saídas JSON, SARIF e HTML
- JSON lido pelo gate

Ponto forte: mostra a regra e o trecho do código. Limite: depende das regras.

Três alertas no código inseguro

```

njsscan: v1.0.0 | Ajin Abraham | opensecurity.in
RULE ID      express_xss
OWASP-WEB    A1: Injection
CWE          CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
DESCRIPTION  Untrusted User Input in Response will result in Reflected Cross Site Scripting Vulnerability.
SEVERITY     ERROR

File         app.js
Match Position 3 - 43
FILES        Line Number(s) 8: 9
Match String  const nome = req.query.nome;
               res.send('<h1>Olá, ' + nome + '</h1>');

RULE ID      generic_os_command_exec
OWASP-WEB    A1: Injection
CWE          CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
DESCRIPTION  User controlled data in 'child_process.exec()' can result in Remote OS Command Execution.
SEVERITY     ERROR

File         app.js
Match Position 3 - 5
FILES        Line Number(s) 12: 14
Match String  exec(req.query.cmd, function (erro, saida) {
               res.type('text').send(erro ? 'Erro' : saida);
               });

RULE ID      eval_nodejs
OWASP-WEB    A1: Injection
CWE          CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
DESCRIPTION  User controlled data in eval() or similar functions may result in Server Side Injection or Remote Code Injection
SEVERITY     ERROR

File         app.js
Match Position 21 - 41
FILES        Line Number(s) 17
Match String  const resultado = eval(req.query.expr);

```

3

ALERTAS CONFIRMADOS

CWE-79 • XSS

CWE-78 • Comando

CWE-95 • eval

Análise dos três achados

Achado	CWE	Por que é real?	Correção
HTML com entrada	79	Entrada chega ao HTML	Responder texto simples
Comando do usuário	78	Entrada chega ao exec	Retirar o comando
Texto como código	95	Entrada chega ao eval	Somar números validados

Os três alertas foram conferidos no código: 0 falsos positivos nessa amostra.

Correção do uso de eval

Antes

```
const resultado =  
  eval(req.query.expr);  
  
res.json({ resultado });
```

Depois

```
const a = Number(req.query.a);  
const b = Number(req.query.b);  
// Validar os números antes  
res.json({ resultado: a + b });
```

O cálculo passa a aceitar números, sem executar um programa enviado pelo usuário.

Correções de XSS e comando

XSS: responder texto simples

```
// Depois da correção  
res.type('text/plain')  
    .send('Olá, ' + nome);
```

O navegador recebe texto e não interpreta a entrada como HTML.

Comando: retirar a execução

```
// Depois da correção  
res.json({  
    mensagem: 'Laboratório do grupo 3'  
});
```

A rota passa a responder uma mensagem fixa, sem chamar exec.

npm audit

Como analisa

- Lê as dependências do lockfile
- Consulta avisos conhecidos
- Classifica a severidade

Uso no pipeline

- Instalar as dependências
- Executar `npm audit --json`
- Bloquear HIGH e CRITICAL

Ponto forte: já faz parte do npm. Limite: depende dos avisos publicados.

Dependência vulnerável encontrada

```
LAB GRUPO 3 - NPM AUDIT x + v
COMANDO: docker compose run --rm lab (npm audit na versão insegura)

# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names - https://github.com/advisories/GHSA-r5fr-rjxr-66jc
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` - https://github.com/advisories/GHSA-f23m-r3pf-42rh
Lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions - https://github.com/advisories/GHSA-xxjr-mmjv-4gpg
fix available via `npm audit fix --force`
Will install lodash@4.18.1, which is outside the stated dependency range

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

CODIGO_DE_SAIDA=1
|
```

1 HIGH

PACOTE VULNERÁVEL

lodash 4.17.15

O pacote aparece em vários avisos. A contagem do resumo é por pacote.

A versão vulnerável foi confirmada; exploração pela rota exige outra análise.

Dependência atualizada e novo teste

Atualização da biblioteca

4.17.15

versão insegura

4.18.1

versão corrigida

```
LAB GRUPO 3 - NPM CORRIGI x + v
COMANDO: docker compose run --rm lab (npm audit na versão corrigida)
found 0 vulnerabilities
CODIGO_DE_SAIDA=0
|
```

0

VULNERABILIDADES CONHECIDAS

Datree

O que fazia

- Analisava YAML de Kubernetes
- Aplicava regras de política
- Rodava antes do deploy

Situação do projeto

- Projeto arquivado
- Sem manutenção contínua
- Pesquisado, mas não executado

Serve para estudo; um projeto novo deve avaliar ferramentas mantidas.

Configuração Kubernetes mais segura

Antes

```
securityContext:  
  privileged: true  
  
# Sem limites de recursos
```

Depois

```
securityContext:  
  privileged: false  
  readOnlyRootFilesystem: true  
  
# Recursos com limites
```

O arquivo corrigido reduz privilégios e define limites. Não houve scan do Datree.

StackHawk e HawkScan

Como funciona

- Testa a aplicação funcionando
- Envia requisições para as rotas
- Observa as respostas

O que precisa

- Endereço autorizado do alvo
- Ambiente de teste disponível
- Login configurado, se necessário

A plataforma possui partes comerciais. Foi pesquisada, mas não executada.

Etapas de um teste DAST

01

Iniciar

Aplicação de teste

02

Configurar

Endereço permitido

03

Executar

Scanner e rotas

04

Revisar

Alertas e contexto

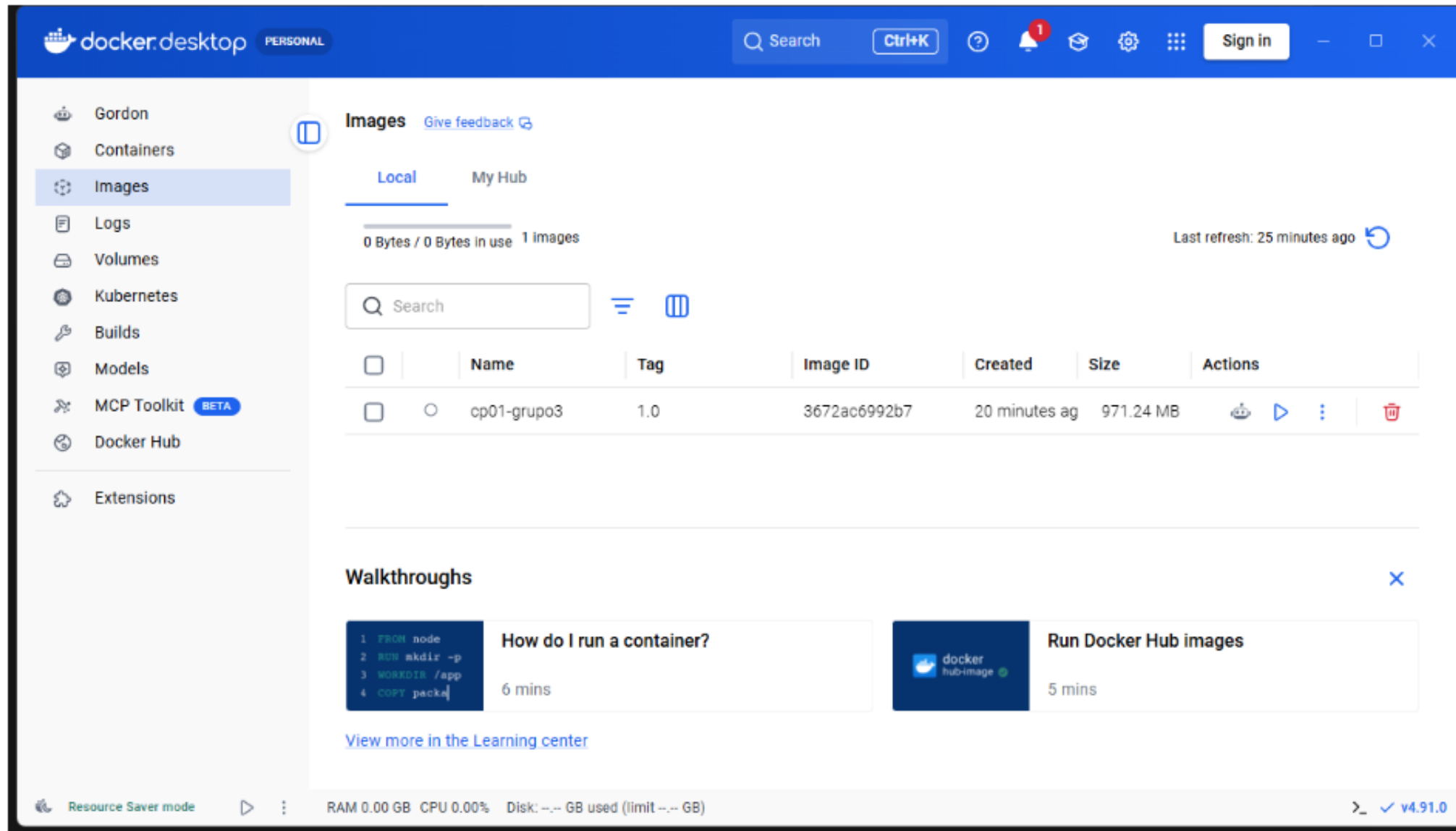
05

Repetir

Depois das correções

Sem acesso às rotas protegidas, o scanner pode testar apenas a tela de login.

Imagem preparada no Docker Desktop



Uma imagem
para os dois
testes
Node.js

Python

njsscan e Semgrep

npm

Versões e imagem do laboratório

```
LAB GRUPO 3 - DOCKER
COMANDO: docker compose build --pull lab

Docker version 29.8.0, build 88096ef
Docker Compose version v5.5.1
WSL: Ubuntu, versão 2

Imagem: cp01-grupo3:1.0
ID: sha256:3672ac6992b77b006867348d9b05d2015cb33269a723ac86014ce11688ef4f25
Criada em: 2026-09-16T18:55:28.743786323Z
Tamanho: 971245040 bytes

RESULTADO: imagem criada com sucesso.
```

Docker Desktop

Compose

WSL 2 / Ubuntu

cp01-grupo3:1.0

Quando o gate bloqueia a entrega

Resultado	Decisão	Código de saída
SAST ERROR	Tratar como HIGH e bloquear	1
SCA HIGH ou CRITICAL	Bloquear	1
Sem alertas nesses níveis	Aprovar	0
Erro ou relatório inválido	Interromper a análise	2

A política está no script: o pipeline usa o código de saída para decidir.

Versão insegura: gate bloqueado

```
LAB GRUPO 3 - TESTE BLOQU  X + v
COMANDO: docker compose run --rm lab inseguro

SAIDA REAL DO CONTAINER:
{
  "alvo": "inseguro",
  "status": "BLOQUEADO",
  "exit_code": 1,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 3,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 1,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  }
}

njsscan-json: 4.809s, código 1
njsscan-sarif: 4.731s, código 1
npm-audit: 0.731s, código 1

CODIGO_DE_SAIDA=1
|
```

3

ALERTAS SAST

1

PACOTE HIGH

Código de saída: 1

Versão corrigida: gate aprovado

LAB GRUPO 3 - TESTE APROVADO

COMANDO: docker compose run --rm lab corrigido

```
{
  "alvo": "corrigido",
  "status": "APROVADO",
  "exit_code": 0,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 0,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 0,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": []
}
```

njsscan-json: terminou em 5.077s, código 0
njsscan-sarif: terminou em 4.825s, código 0
npm-audit: terminou em 0.655s, código 0
|

0

ALERTAS SAST

0

PACOTES VULNERÁVEIS

Código de saída: 0

GitHub Actions: bloqueio confirmado

The screenshot shows the GitHub Actions interface for a repository named 'cp01-cloud-security-grupo-3'. The workflow 'Segurança do grupo 3 #5' is displayed, which was manually triggered 6 minutes ago. The status is 'Failure' with a total duration of 1m 3s and 1 artifact. The workflow file 'seguranca.yml' is shown, triggered on 'workflow_dispatch'. A summary table indicates the results of the jobs: 'seguranca' failed (34s), 'politica' passed (6s), and 'docker' passed (58s). The left sidebar shows the workflow structure with 'seguranca' marked as failed and 'politica' and 'docker' as successful. The bottom section is labeled 'Annotations'.

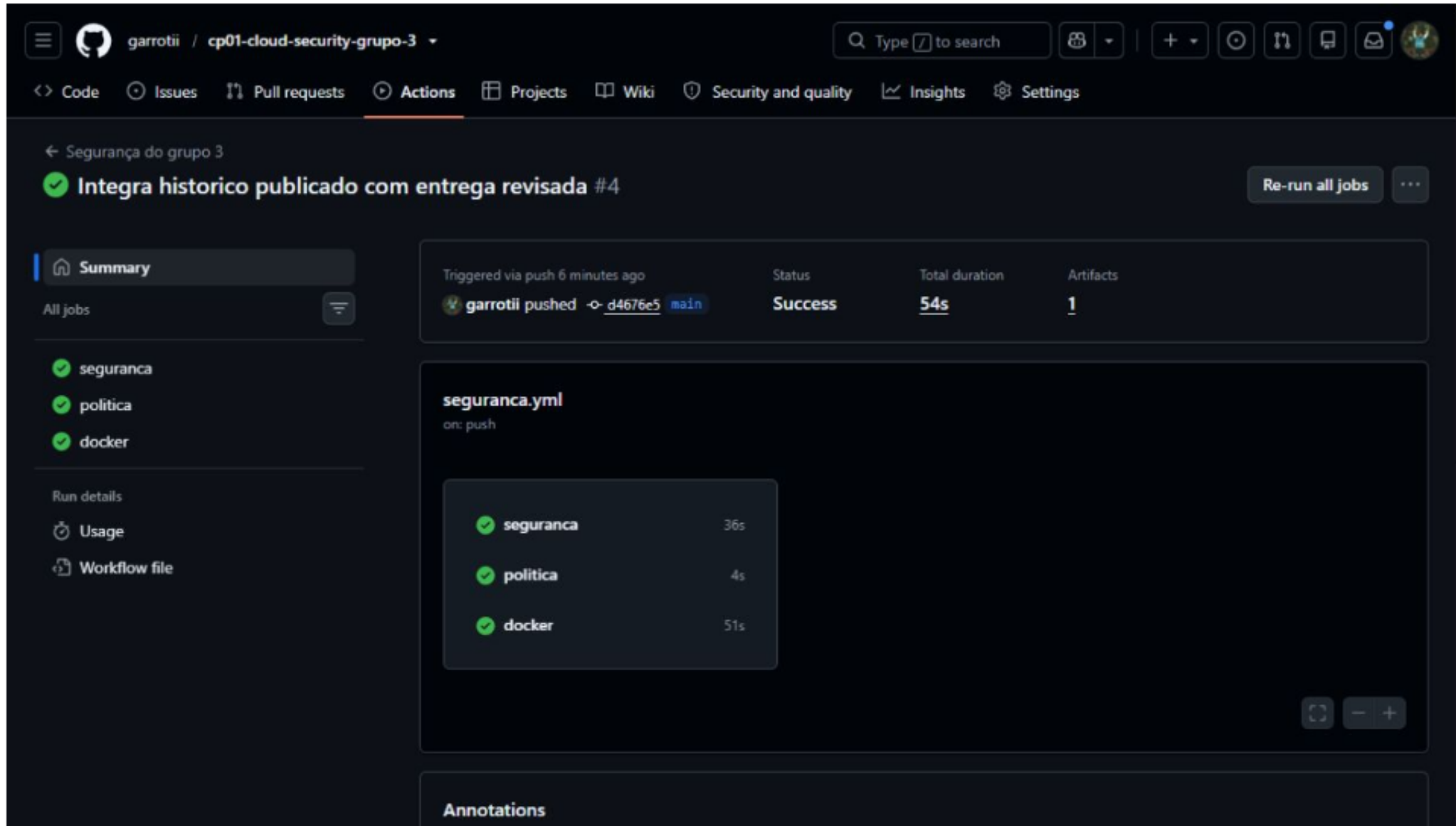
Job	Status	Duration
seguranca	Failure	34s
politica	Success	6s
docker	Success	58s

Falha esperada

A versão insegura foi bloqueada.

Os relatórios ficaram anexados à execução.

GitHub Actions: correção aprovada



The screenshot shows the GitHub Actions interface for a repository named 'cp01-cloud-security-grupo-3'. The workflow 'Integra historico publicado com entrega revisada #4' is shown as successful. The workflow file 'seguranca.yml' is triggered on push. The run details show three jobs: 'seguranca' (36s), 'politica' (4s), and 'docker' (51s), all of which passed. The total duration of the run is 54s. The interface includes a sidebar with navigation links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The main content area displays the workflow status and details.

Triggered via push 6 minutes ago	Status	Total duration	Artifacts
garrotii pushed -o d4676e5 main	Success	54s	1

Job	Duration
seguranca	36s
politica	4s
docker	51s

Teste aprovado

A versão corrigida passou.

Os dois cenários usam o mesmo commit.

Comparação das ferramentas

Critério	njsscan	npm audit	Datree	StackHawk
Entrada	Código	Dependências	Kubernetes	Aplicação ativa
Situação no lab	Executado	Executado	Pesquisado	Pesquisado
Resultado	3 alertas → 0	1 HIGH → 0	Sem execução	Sem execução
Limitação	Regras	Avisos conhecidos	Arquivado	Cobertura e conta

SAST e SCA desde o começo; IaC antes do deploy e DAST no ambiente de teste.

Laboratório guiado: comandos e perguntas

01 Executar a versão insegura

```
docker compose run --rm lab inseguro
```

02 Comparar código e dependências

Conferir XSS, exec, eval e a versão do lodash.

03 Executar a versão corrigida

```
docker compose run --rm lab corrigido
```

Perguntas: como ERROR é tratado pelo gate? Qual CWE corresponde ao eval?

O que o laboratório mostrou

Aprendizado principal

Código e bibliotecas têm problemas diferentes e precisam de correções diferentes.

Limites do resultado

- A amostra tem falhas intencionais
- Datree e StackHawk não rodaram
- Zero alertas não garante segurança
- A revisão humana continua necessária

O scanner aponta o alerta; a equipe entende a causa, corrige e testa novamente.

Referências principais e perguntas

Documentação e projetos

- njsscan: repositório e documentação
- npm: audit e SBOM
- Datree: repositório e situação do projeto
- StackHawk: HawkScan e configuração

Fundamentos e análise

- NIST SP 800-218 / SSDF
- MITRE CWE-78, CWE-79 e CWE-95
- GitHub Advisory Database
- Relatórios reais do laboratório

Perguntas? O relatório e o LAB.md estão no repositório público do grupo 3.